Valid Signatures, Stale Indexes: A Controlled Chunk-Identity Audit of RAG Source Revocation

Ezekiel Ologunde

Abstract

Signed provenance does not by itself ensure that a mutable retrieval index points to the record whose signature is verified. We audit one pinned RAGtrap source-revocation implementation using an eight-case matrix of repeated versus fresh chunk identities, same versus different sources, and unchanged versus changed text. The original accepts every second ingestion. Two cross-source identity-reuse cases leave stale source memberships, so revoking the former source removes the replacement source's current passage. Two changed-text reuse cases leave an old hash bucket resolving to a record containing a different content hash. All current record signatures remain valid. An immutable-identity overlay rejects the four reused-ID cases and preserves the four fresh-ID controls. A separately recorded post-hoc upsert repair accepts all eight and maintains index consistency in this single-threaded matrix. Eight checks pass; the original offline suite has 32 passes and one optional dependency skip. These are controlled API counterexamples, not production incidence, cryptographic attacks or a replication of the paper's full experiments. The contribution is a revision-specific implementation audit and explicit update contract; standalone novelty remains unestablished.

Keywords

RAG, source revocation, provenance, index consistency, reproducibility

1 Prior Work and Question

RAGtrap records signed passage provenance, indexes it by content hash and source, and removes passages from a source known to be compromised [1]. It also refuses future ingestion from a revoked source. Its authors explicitly position it as a recovery layer, distinguish exact-hash limitations from poisoning detection, and identify the reference datastore as in-memory. Signed attribution and source-specific revocation are therefore prior capabilities, not our claimed invention.

We examine a narrower boundary: what happens when an already accepted chunk identifier is reused? A signature can remain valid even when an index points to a newer record that no longer belongs to the indexed source or hash. We ask whether current record ownership and hash binding survive two ordinary ingestion calls, and which admission/update contract preserves them. Maintaining secondary indexes is a standard systems requirement; our candidate contribution is an executable counterexample and bounded repair against one public artifact.

2 Pinned Artifact and Design

We pin `CristhianKapelinski/sbseg2026-ragtrap` at commit `c11da0ba`…and acquire 32 source/document files with SHA-256 receipts. The repository declares MIT licensing. We also acquire the author's frozen 337,536-byte BEIR/nq sample; the source's sample-integrity check verifies its digest. Reading copies and acquired upstream material remain separate from our authored package.

Table 1: Eight cases per variant. P: source-index mismatch; H: hash-index mismatch; F: removal of a current other-source record. Counts describe cases, not prevalence. The upsert comparator is post-hoc.

Variant	Accepted	P	H	F
Original	8	2	2	2
Immutable IDs	4	0	0	0
Consistent upsert (post-hoc)	8	0	0	0

The original `put` checks source revocation, replaces the primary record/chunk dictionaries, and adds memberships to source/hash indexes. For a reused identifier, it does not remove previous memberships first. The gate's character splitter constructs identifiers as `document_id::c<i>`, so repeating a document identifier reaches the same path through the ordinary gate. No inconsistent record, forged signature or direct index mutation is needed in our fixture.

After inspecting these paths and the relevant original tests, we record a protocol before scoring. The design enumerates all eight combinations of same/different chunk ID, principal and text for two sequential ingestions. Each case uses a fresh Ed25519 signer and controlled local text. We inspect index membership against the current record, verify its signature, look up the former text, and revoke the former source. We count a cross-source false purge only when a record currently owned by another source is removed. No model, network sink, malicious document or external service participates.

This is exploratory source auditing followed by a locally frozen counterexample, not blinded discovery or external preregistration. It identifies accepted API behavior; external caller authorization and global identifier allocation remain assumptions outside this test.

3 Controlled Results

Table 1 summarizes the matrix. The original accepts all eight second ingestions. Two reused-ID cases with different principals leave stale source memberships. Revoking the former principal then removes the replacement principal's current record. Two reused-ID cases with changed text leave an old hash membership that resolves to a current record with a different content hash. In the changed-source/changed-text case, traceback attributes the former text to the replacement source after verifying the replacement record's valid signature.

All live record signatures verify in every variant. Thus the failure is a mismatch between index membership and current record binding, not a signature forgery. Fresh-ID cases provide a negative control: source-specific removal and exact-hash binding behave as

expected. Repeated identical text under one source also avoids the differing-membership condition.

The source's original offline suite runs without source modification: 32 tests pass and one optional false-purge contrast is skipped because pyarrow is absent. That contrast and the full author experiments are not claimed as reproduced. Our runtime is Python 3.12.14, cryptography 50.0.2 and pytest 8.4.2, rather than the author's complete uv-locked environment. The signature/index counterexample does not depend on a timing measurement.

4 Two Explicit Update Contracts

The primary overlay makes passage identities immutable. It rejects an identifier already present before changing any store state, validates chunk/record identity and per-chunk hash agreement, and retains the existing source-denial check. It accepts all four fresh-ID cases and rejects the four reused-ID cases. Its zero observed consistency failures therefore come with a restriction: an updated passage must receive a new versioned identity. This is not transparent upsert.

After observing the primary matrix, we add an explicitly post-hoc comparator. It validates admission first, removes previous memberships using the original removal operation, then inserts the new current record. It accepts all eight cases and has no observed index mismatch or cross-source false purge on this same matrix. This distinguishes consistency repair from the immutable overlay's rejection of updates; it is not a second held-out efficacy result.

Neither contract establishes permission to transfer an identifier between sources. The upsert path assumes that the caller has already authorized the replacement; the immutable path requires caller-managed version identities. Neither adds concurrency control, durable storage, transactional vector-database updates, cache invalidation or migration of an already inconsistent store. Existing stale state must be separately validated or rebuilt.

5 Verification and Limits

Eight checks cover the valid-signature counterexample, the ordinary splitter/gate path, fresh-version preservation, rejection without mutation, record/hash agreement, continued source-denial behavior, matrix availability and post-hoc upsert consistency. Every original source byte is checked before running the matrix; case-level outcomes and both protocols are retained.

The result is a revision-specific accepted-input failure condition. It does not establish how frequently real deployments reuse IDs, whether an attacker can reach trusted admission, or whether the paper's unique-ID experiments contain false purges. We do not contradict the reported unique-ID benchmark results or claim a new poisoning defense. A contract that required globally unique immutable IDs would exclude this condition; the implementation can enforce that precondition rather than relying on callers silently.

The audit also does not address physical erasure, parametric unlearning, revoked information in unmanaged copies or restoration of authority. Those require different experiments. Source revocation, signature validity, current index consistency and external authorization are separate properties.

Our authored reproducibility package contains acquisition scripts, locally recorded protocols, the overlay/comparator and case-level results. Original code and sample bytes are separately reacquired. The ACM source and Overleaf package retain class-license notices. Independent human assessment, author contact, public submission and peer review have not occurred. Generative AI assisted source inspection, implementation and drafting. A broader independently reviewed artifact study would be needed to support a stronger general claim.

References

[1] Cristhian Eduardo Kapelinski de Avila and Diego Kreutz. 2026. RAGtrap: Source Revocation and Indexed Provenance Lookup for Poisoned RAG Corpora. SBSeg 2026, WTICG. Author artifact: https://github.com/CristhianKapelinski/sbseg2026-ragtrap.